

# Assignment 5: Introduction to Relational Databases

The purpose of this assignment is to practice writing server-side applications that read from and write to a database, using the SQL language and SQLite database management system. You will create a simple web application for browsing and rating books.

The main part of the assignment will be creating a basic REST API for books; a REST API is a server-side application that provides an interface (through URL endpoints) for client applications to interact with the book data. I've provided a basic client application (just an HTML page) that you can use to test your REST API.

Throughout this assignment, use **prepared statements with placeholders** to make database queries, sanitizing all parameters that come from the client.

## Task 0: Setup

Create a new GitHub Codespace with the blank template. Initialize a Node.js project, and create two subfolders: **public**, and **database**.

In the starter files I've provided the following:

- a database file called **longlist-advance.db** with tables of data related to famous books. Move this into the **database** folder. Create a new file in this folder called **db.js**; have it open a connect to the database, and export it.
  - o Study the database using the **SQLite Viewer** extension for VS Code.
- a file called **index.html** with a webpage for testing your server-side application. Move this to the **public** folder.

Create an Express application in a file called **app.js** in the root of the project folder. It should import the **database object** from **/database/db.js** and also serve static files from the **public** folder.

Use the **index.html** file throughout the assignment to test your work. You can also test the URL endpoints directly by typing them into the address bar of the browser.

## Task 1: Authors

Create a route handler for **GET** requests to the path **/api/v1/authors**. It should select all **columns of all rows** from the **authors** table and send them to the client as a **JSON** object.

Use the **Browse Authors** hyperlink on the test page to test this. The result should look something like this in Firefox (You may see different books, as rows are not guaranteed to be fetched in the same order every time):

| JSON     | Raw Data              | Headers                        |
|----------|-----------------------|--------------------------------|
| Save     | Copy                  | Collapse All Expand All Filter |
| ▼ 0:     |                       |                                |
| id:      | 1                     |                                |
| name:    | "Adania Shibli"       |                                |
| country: | "Palestine"           |                                |
| birth:   | 1974                  |                                |
| ▼ 1:     |                       |                                |
| id:      | 2                     |                                |
| name:    | "Ahmed Saadawi"       |                                |
| country: | "Iraq"                |                                |
| birth:   | 1973                  |                                |
| ▼ 2:     |                       |                                |
| id:      | 3                     |                                |
| name:    | "Alia Trabucco Zerán" |                                |
| country: | "Chile"               |                                |
| birth:   | 1983                  |                                |
| ▼ 3:     |                       |                                |
| id:      | 4                     |                                |
| name:    | "Amanda Svensson"     |                                |
| country: | "Sweden"              |                                |

## Task 2: Books

Create a route handler for **GET** requests to the path **/api/v1/books**. Start by having it fetch the **id, isbn, title, format, and year** columns for **all rows** from the **books** table and send them to the client as a JSON object. But then add the following functionality:

- We'd like to set up **paging** functionality to break up responses into smaller pages that each have **10** books. The page number will be specified as a **query parameter** in the URL. For example, if the client makes a request to **/api/v1/books?page=2**, they will receive only 10 books, the 30<sup>th</sup> to 39<sup>th</sup> books in the database. (The first page, with books 0 to 9, is page=0.) To do this, follow these steps:

- Limit the number of rows fetched from the query by using the **LIMIT** keyword. The query should return only **10** rows. Reference: [https://www.w3schools.com/mysql/mysql\\_limit.asp](https://www.w3schools.com/mysql/mysql_limit.asp)
- Skip the first **N** books in the database by using the **OFFSET** keyword. Test this with an **offset of 10**. Reference same as previous: [https://www.w3schools.com/mysql/mysql\\_limit.asp](https://www.w3schools.com/mysql/mysql_limit.asp)
- Get the **query parameter** of the request using **req.query.page**. If no query parameter is provided, the default should be **page=0**. Use the **query parameter** to figure out what the **OFFSET** should be for the query.
- Use the **?** placeholder in the query to insert the correct offset into the **prepared statement**.
- We'd also like to allow the client to specify whether they'd like to get books in the **hardcover** format or **paperback** format. First, use a **WHERE** keyword to filter the results using the **format** column. Test this by getting only **hardcover** results.
- Use a query parameter called **format** to allow the client to specify a desired format. Insert the parameter into the prepared statement using a **?** placeholder.
  - If no value for **format** is provided, the default value should be **format=hardcover**.
- Test this using the **Browse Books** form on the test page; note the query parameters in the URL:

The screenshot shows a web browser with the address bar displaying `api/v1/books?format=paperback&page=2`. Below the browser, a REST client interface is visible, showing the JSON response for the GET request. The response is an array of 4 book objects, indexed 0 to 3. The third object (index 2) is highlighted in blue.

```

{
  "0": {
    "id": 23,
    "isbn": "9781999992859",
    "title": "After the Sun",
    "format": "paperback",
    "year": 2022
  },
  "1": {
    "id": 24,
    "isbn": "9781911284611",
    "title": "Tomb of Sand",
    "format": "paperback",
    "year": 2022
  },
  "2": {
    "id": 25,
    "isbn": "9781910695593",
    "title": "The Books of Jacob",
    "format": "paperback",
    "year": 2022
  },
  "3": {
    "id": 26,
    "isbn": "9781916277182",
    "title": "Curse of Bunny",
    "format": "hardcover",
    "year": 2022
  }
}

```

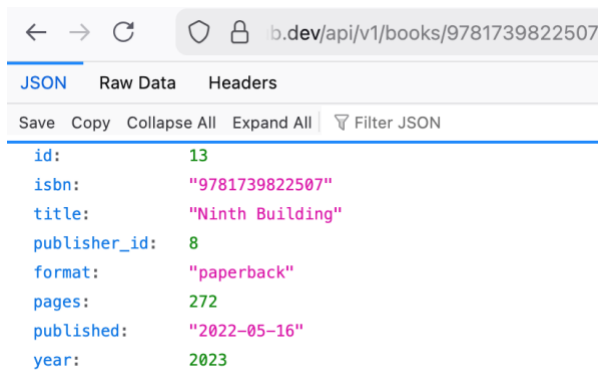
## Task 3: Specific Books

We have a way for clients to request large collections of books with just a few details about each book. Now, we'd like a way for clients to request data about just one book, using its ISBN number as an identifier, but with more complete data about that book.

Set up a route handler for **GET** requests to **/api/v1/books/:isbn**

It should send ALL columns for ONE row, fetched using the ISBN number.

For example, a GET request to **/api/v1/books/9781739822507** should send a response that looks like this:



You can use the **Details for Selected Books** part of the test page to test this.

## Task 4: Filling In Missing Data

As you can see, each book stores a **Publisher ID number**, but not the name of the publisher. You can cross-reference this ID with the **publishers** table using **SQLite Viewer**:

| id | publisher               |
|----|-------------------------|
| 1  | And Other Stories       |
| 2  | Charco Press            |
| 3  | Europa Editions         |
| 4  | Faber & Faber           |
| 5  | Fitzcarraldo Editions   |
| 6  | Granta Books            |
| 7  | Harvill Secker          |
| 8  | Honford Star            |
| 9  | Jonathan Cape           |
| 10 | Literatura Pandom House |

So the publisher of the book from the example in Task 3, with **publisher\_id = 8**, is **Honford Star**.

Modify the route handler from Task 3 to make a **second query**, with a second **prepared statement**, to the **publishers** table to get the **name of the publisher using its ID**.

(There's a way to do this with a single query, but we'll learn about that next week.)

Add this publisher's name to the object that you send back to the client. Your response should now look like this:

| JSON                      | Raw Data | Headers                 |
|---------------------------|----------|-------------------------|
| Save                      | Copy     | Collapse All Expand All |
| id: 13                    |          |                         |
| isbn: "9781739822507"     |          |                         |
| title: "Ninth Building"   |          |                         |
| publisher_id: 8           |          |                         |
| format: "paperback"       |          |                         |
| pages: 272                |          |                         |
| published: "2022-05-16"   |          |                         |
| year: 2023                |          |                         |
| publisher: "Honford Star" |          |                         |

## Task 5: Book Ratings

You may have noticed that our database has a table called **ratings** that lists all ratings each book has received from users. Each row in this table corresponds to one rating from a user, and has a column for **book\_id**, storing the ID of the book to which the rating applies, and a column called **rating** that has the actual numerical rating between 0 and 5.

There are more than 600,000 ratings in this table and only 78 books in the database, meaning each book has many, many ratings. We'd like to modify the route handler from Task 3 to include the **average rating** of the book in the response. To do this, implement a **third query**, this time to the **ratings** table, to get all ratings for the current book. Use a JavaScript **forEach** loop to calculate the **average** of all ratings, and add the average to the response object. (There's a way to calculate averages using SQL queries, but that's beyond the scope of this course so you can use JavaScript instead.) The response should now look like this:

| JSON            | Raw Data          | Headers                             |
|-----------------|-------------------|-------------------------------------|
| Save            | Copy              | Collapse All Expand All Filter JSON |
|                 |                   |                                     |
| id:             | 13                |                                     |
| isbn:           | "9781739822507"   |                                     |
| title:          | "Ninth Building"  |                                     |
| publisher_id:   | 8                 |                                     |
| format:         | "paperback"       |                                     |
| pages:          | 272               |                                     |
| published:      | "2022-05-16"      |                                     |
| year:           | 2023              |                                     |
| publisher:      | "Honford Star"    |                                     |
| average_rating: | 3.775000000000002 |                                     |

## Task 6: Submitting New Ratings

As you can see, there's one more form on the test page: this form allows one to submit a new rating, between 0 and 5, for a chosen book. (We're choosing the book by its ID here for simplicity, not its ISBN.)

Implement a route handler for **POST** requests that come from this form. It should simply **INSERT** a new row to the **ratings** table. To test it, try inserting a few ratings for the book with **ID=13** and notice how the average rating changes. For example, after submitting three 0-star reviews (I don't hate this book, I'm just testing the program!) the average rating now looks like this:

| JSON            | Raw Data           | Headers      |
|-----------------|--------------------|--------------|
| Save            | Copy               | Collapse All |
| Expand All      | Filter JSON        |              |
| <hr/>           |                    |              |
| id:             | 13                 |              |
| isbn:           | "9781739822507"    |              |
| title:          | "Ninth Building"   |              |
| publisher_id:   | 8                  |              |
| format:         | "paperback"        |              |
| pages:          | 272                |              |
| published:      | "2022-05-16"       |              |
| year:           | 2023               |              |
| publisher:      | "Honford Star"     |              |
| average_rating: | 3.5116279069767464 |              |

## Grading

- Task 0
  - [1 mark] project set up correctly with static files and database folder;
- Task 1
  - [1 mark] route handler sends author data;
- Task 2
  - [1 mark] Paging set up for books with query parameters and placeholders in prepared statement;
  - [1 mark] Query parameter for format and statement placeholder;
- Task 3
  - [1 mark] All fields for specific book sent;
- Task 4
  - [1 mark] Publisher name added to response;
- Task 5
  - [1 mark] Average rating calculated and added to response;
- Task 5
  - [1 mark] Form for adding a rating implemented.

Total: 8

As usual, up to -25% may be deducted for improper hand in, poor organization, etc.

## Hand In

Remove the **node\_modules** folder and any other unnecessary files, like these instructions. (Note that **package.json** IS necessary.) Rename the project folder to **a5-firstname-lastname**, zip the folder, and submit to Brightspace.